

Spring Bean Lifecycle & Scopes

1 What is a Spring Bean?

A **Spring Bean** is an object that is created, configured, and managed by the Spring IoC container (`ApplicationContext`).

```
@Service
public class UserService {
}
```

Spring creates and manages the `UserService` bean instance throughout its lifecycle.

2 Bean Lifecycle

The simplified sequence from container initialization to shutdown:

Spring Container starts → Create Bean → Dependency Injection → **@PostConstruct** → Bean Ready → Application Runs → Container Shutdown → **@PreDestroy** → Bean Destroyed

```
@Service
public class UserService {

    @PostConstruct
    public void init() {
        System.out.println("Bean initialized");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Bean destroyed");
    }
}
```

INTERVIEW ANSWER

"During the Spring Bean lifecycle, the container creates the bean, injects its dependencies, performs initialization callbacks such as `@PostConstruct`, and when the container shuts down, performs destruction callbacks such as `@PreDestroy`."

3 Important Lifecycle Hooks

@PostConstruct: Runs after dependency injection is complete and before the bean is put into service.

- Initialize resources
- Perform startup logic
- Validate configuration

@PreDestroy: Runs before the bean is destroyed during container shutdown.

- Release resources
- Close database/network connections
- Cleanup tasks

4 Bean Scopes

A scope defines how long a bean instance lives and how many instances Spring creates.

Main scopes: `Singleton`, `Prototype`, `Request`, `Session`, `Application`, `WebSocket`.

*For backend engineering interviews, **Singleton**, **Prototype**, and **Request** are primary focal points.*

5 Singleton Scope

Default Spring scope. Spring creates **one instance per Spring IoC container**.

```
@Service
@Scope("singleton")
public class UserService {
}
```

Important: Singleton does not mean one object per JVM—it means one instance per `ApplicationContext`.
Singleton beans are **NOT automatically thread-safe**. Avoid storing mutable request-specific state!

Bad Practice (Shared state):

```
@Service
public class UserService {
    private User currentUser; // ❌ Shared mutable state across threads
}
```

Best Practice (Stateless / Local variables):

```
@Service
public class UserService {
    public User getUser(Long id) {
        User user = ...; // Local variable -> thread-safe
        return user;
    }
}
```

6 Prototype Scope

```
@Component
@Scope("prototype")
public class ReportGenerator {
}
```

Spring creates a new instance **every time** the bean is requested from the container (e.g., via `getBean()`).

Important Interview Point: Spring manages the creation and configuration of prototype beans, but after delivering them to the caller, Spring does **not** manage their complete destruction lifecycle (e.g., `@PreDestroy` may not execute automatically).

7 Request Scope

```
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST)
public class RequestContext {
}
```

A new instance is created for each HTTP request and destroyed when the request completes. Used for request-specific metadata, User Context, or Request IDs.

8 Scope Comparison Table

Scope	Instances Created	Lifetime
Singleton	One per <code>ApplicationContext</code>	Application / Container lifecycle
Prototype	New instance when requested	Until garbage collected (Container doesn't manage cleanup)
Request	One per HTTP request	Duration of the HTTP request
Session	One per HTTP session	Duration of the HTTP session

9-10 Scope Injection Mismatches

Q9: What happens when a Prototype bean is injected into a Singleton bean?

The prototype bean is resolved **only once** when the singleton bean is instantiated at container startup. Consequently, the singleton will continue using the *same* prototype instance.

Solution: Use `ObjectProvider<T>`, `Provider<T>`, or `@Lookup`.

```
private final ObjectProvider<ReportGenerator> provider;

public ReportGenerator createReportGenerator() {
    return provider.getObject(); // Fetches a new prototype instance each call
}
```

Q10: What happens when a Request-scoped bean is injected into a Singleton bean?

Direct injection causes a scope mismatch because the singleton outlives individual HTTP requests. To solve this, Spring uses a **Scoped Proxy**:

```
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST, proxyMode =
    ScopedProxyMode.TARGET_CLASS)
public class RequestContext { }
```

The singleton receives a lightweight proxy, which dynamically delegates method calls to the active request's bean instance.

11 Bean Lifecycle vs. Bean Scope

Lifecycle: Answers *"What steps does a bean go through from creation to destruction?"* (Create → Inject → Initialize → Use → Destroy).

Scope: Answers *"How many instances exist and how long do they live?"* (Singleton, Prototype, Request, Session).

12 Master Interview Answer & Checklist

"A Spring Bean is managed by the Spring IoC container. During its lifecycle, Spring creates the bean, injects its dependencies, performs initialization such as **@PostConstruct**, and executes destruction callbacks such as **@PreDestroy** when the container shuts down. Bean scope determines the number and lifetime of bean instances. **Singleton** is the default and creates one instance per `ApplicationContext`, **Prototype** creates a new instance whenever requested, and **Request** scope creates one instance per HTTP request."

Must-Know Rapid Fire Review:

- **Default Scope:** Singleton.
- **Singleton = JVM Scope?** No, 1 per `ApplicationContext`.
- **Thread Safety:** Singletons are NOT automatically thread-safe.
- **@PostConstruct:** Post-dependency injection initialization.
- **@PreDestroy:** Cleanup execution prior to shutdown.
- **Prototype Destruction:** Not completely managed by Spring.
- **Scoped Proxy Necessity:** Enables injecting shorter-lived beans (e.g., Request) into longer-lived beans (e.g., Singleton).